[SOLUTION TEMPLATE] Assignment 1: Imitation Learning

提交截止日期见课程站点

提交入口：`https://oc.sjtu.edu.cn/courses/95065`

姓名：潘昊辰　学号：523120910233

1 Introduction

The goal of this assignment is to experiment with imitation learning, including direct **behavior cloning** and the **DAgger** algorithm. The methods we learn about in this assignment will lay the groundwork for subsequent assignments where we explore more advanced reinforcement learning techniques.

In **Behavior Cloning (BC)**, we treat imitation learning as a supervised learning problem: given a dataset of expert demonstrations $\mathcal{D} = \{(s_i, a_i)\}$, we train a policy π_θ to minimize:

$$\min_\theta \mathbb{E}_{(s,a)\sim\mathcal{D}} \left[\|\pi_\theta(s) - a\|^2 \right]$$

(or the equivalent negative log-likelihood loss for a stochastic policy).

In **DAgger (Dataset Aggregation)**, we iteratively:

1. Use the current policy π_θ to roll out trajectories in the environment;
2. Query the expert policy to relabel the visited states with expert actions;
3. Aggregate the new data into the training set and retrain π_θ.

This procedure addresses the *distribution shift* problem inherent in behavior cloning, where the learner encounters states outside the expert's demonstration distribution.

1.1 Code

Files to edit (fill in `TODO` sections):

- `sjtuRL2026/policies/MLP_policy.py` — MLP policy: `forward()` and `update()`.
- `sjtuRL2026/infrastructure/utils.py` — `sample_trajectory()`: collect a single rollout.
- `sjtuRL2026/scripts/run_hw1.py` — Training loop:
  - DAgger data collection: `utils.sample_trajectories()`
  - Expert relabeling of collected observations
  - Sampling from replay buffer for gradient updates

1.2 Running the Code

Behavior Cloning example (Ant):

```
python sjtuRL2026/scripts/run_hw1.py \
  --expert_policy_file sjtuRL2026/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name bc_ant --n_iter 1 \
  --expert_data sjtuRL2026/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1
```

DAgger example (Ant, 10 iterations):

```
python sjtuRL2026/scripts/run_hw1.py \
  --expert_policy_file sjtuRL2026/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name dagger_ant --n_iter 10 \
  --do_dagger \
  --expert_data sjtuRL2026/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1
```

Available environments: `Ant-v4`, `Walker2d-v4`, `HalfCheetah-v4`, `Hopper-v4`.

Tip: use `--video_log_freq -1` during debugging to disable video logging.

2 Section 1: Behavior Cloning

2.1 BC Results Table

- Run BC (`n_iter=1`) on at least one environment.
- Report the mean and standard deviation of evaluation returns (`Eval_AverageReturn`, `Eval_StdReturn`).
- Include: environment name, `ep_len`, `eval_batch_size`, network architecture (layers × hidden units), training steps, expert data size.
- Note: ensure `eval_batch_size` > `ep_len` so that multiple trajectories are evaluated, making mean/std meaningful.

Solution:

环境	评估平均回报	评估回报标准差	专家平均回报
Ant-v4	976.30	195.69	4681.89
HalfCheetah-v4	3236.47	93.13	4034.80

Table 1: BC 在两个环境上的实验结果。网络用了 2 层隐藏层 × 64 单元，激活函数 Tanh，学习率 5e-3，训练了 1000 个梯度步，batch size 100。

实验设置：`ep_len`=1000，`eval_batch_size`=5000（每次评估采 5 条轨迹，所以报告的标准差是有意义的），`batch_size`=1000，每个环境用的专家数据是 2 条轨迹（共 2000 个转移），`seed`=1。

BC 在两个环境上的表现都没追上专家。Ant 上大概只有专家回报的 21%，HalfCheetah 好一些，约在 80% 左右。差距主要是分布漂移（distribution shift）造成的：测试时策略遇到的是专家演示里没出现过的新状态，误差会沿着整条轨迹一步步放大。Ant 上这个问题更严重，因为它的动力学更复杂，智能体很容易滑到没法恢复的局面。

2.2 Hyperparameter Experiment

- Choose at least one hyperparameter that significantly affects BC performance. Possible choices include:
  - Number of training gradient steps (`num_agent_train_steps_per_iter`)
  - Amount of expert data (subset vs. full)
  - Network architecture (number of layers, hidden layer width)
- Keep all other settings fixed and vary only the chosen hyperparameter on the **same** environment.
- Plot "BC performance (`Eval_AverageReturn`) vs. chosen hyperparameter".
- In the caption, describe:
  - Which hyperparameter was varied and its range
  - Why this hyperparameter affects BC performance
  - Other key settings (environment, `ep_len`, `eval_batch_size`, etc.)

Solution:

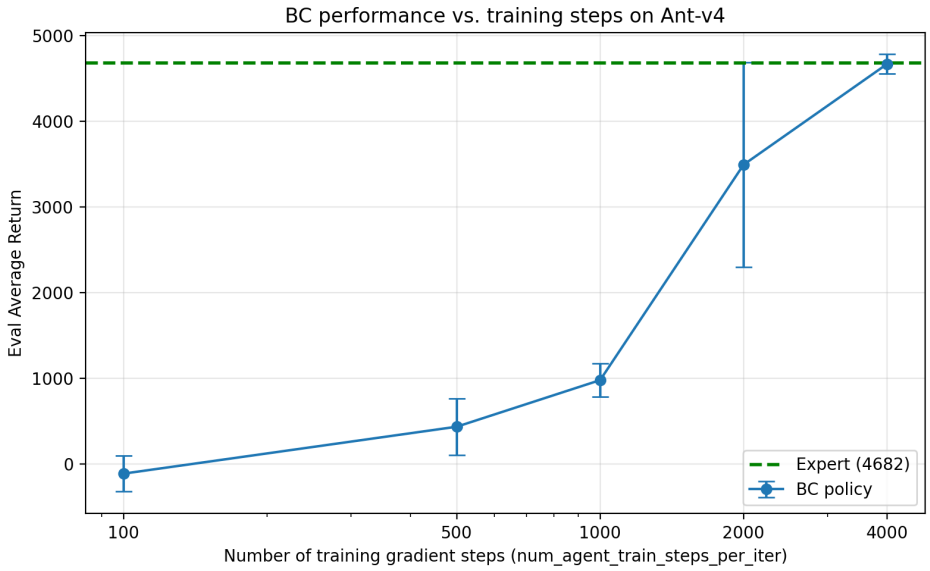


Figure 1: BC 在 Ant-v4 上随训练梯度步数的变化。步数取了 $\{100, 500, 1000, 2000, 4000\}$ 这几个值，其他设置都没动：`ep_len`=1000，`eval_batch_size`=5000（测了 5 条轨迹），网络是 2×64 的 Tanh，学习率 5e-3，训练 batch size 100，专家数据用了 2 条轨迹（2000 个转移）。步数越多，策略对专家数据拟合得越好，回报也跟着往上走；到 4000 步的时候已经离专家很近了（4667.79 对 4681.89）。误差棒对应的是 5 条评估轨迹的 Eval_StdReturn。

下面几组命令每次只改 `--num_agent_train_steps_per_iter` 的值，分别跑了 100、500、1000、2000、4000 这几个步数：

```
python sjtuRL2026/scripts/run_hw1.py \
  --expert_policy_file sjtuRL2026/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name bc_ant_steps<N> --n_iter 1 \
  --expert_data sjtuRL2026/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1 --eval_batch_size 5000 \
  --num_agent_train_steps_per_iter <N>
```

3 Section 2: DAgger

3.1 DAgger Learning Curve

- Run DAgger (`--do_dagger --n_iter 10` or more) on at least one environment (preferably the same as your BC experiment).
- Plot "DAgger iteration vs. `Eval_AverageReturn`" with error bars (`Eval_StdReturn`):
  - x-axis: DAgger iteration number (0 = pure BC warmstart, 1, 2, …)
  - y-axis: `Eval_AverageReturn`
- On the same plot, include horizontal baselines:
  - **Expert policy** performance (dashed green line)
  - **BC-only** performance (dashed red line, i.e., iteration 0)
- In the caption, state: environment, network architecture, `ep_len`, `eval_batch_size`, expert data size, and other relevant hyperparameters.

Solution:

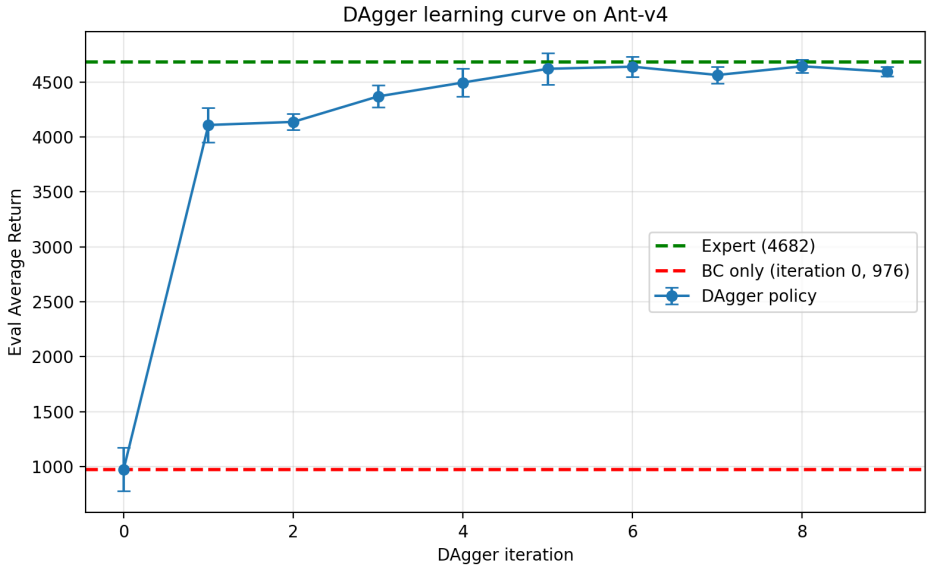


Figure 2: DAgger 在 Ant-v4 上的学习曲线。横轴是迭代轮数，第 0 轮是纯 BC 预热（只用专家数据），之后 1~9 轮是 DAgger 迭代。纵轴是 Eval_AverageReturn，误差棒来自 Eval_StdReturn。绿色虚线是专家策略的平均回报（4681.89），红色虚线是 BC-only 基线（976.30）。实验设置和前面一致：`ep_len`=1000，`eval_batch_size`=5000（5 条轨迹），网络 2×64（Tanh），学习率 5e-3，训练 batch size 100，专家数据 2 条轨迹（2000 个转移），每轮 `num_agent_train_steps_per_iter`=1000。

完整的命令如下：

```
python sjtuRL2026/scripts/run_hw1.py \
  --expert_policy_file sjtuRL2026/policies/experts/Ant.pkl \
  --env_name Ant-v4 --exp_name dagger_ant_final --n_iter 10 --do_dagger \
  --expert_data sjtuRL2026/expert_data/expert_data_Ant-v4.pkl \
  --video_log_freq -1 --eval_batch_size 5000
```

3.2 BC vs. DAgger Comparison

- Briefly discuss:
  - How does DAgger compare to BC-only?
  - Does DAgger approach expert-level performance? After how many iterations?
  - What is the effect of distribution shift, and how does DAgger address it?

Solution:

DAgger 与 BC-only 的对比。只跑 BC 的话，Ant 上的回报只有 976.30，大概是专家的 21%。DAgger 的效果明显好很多：第一轮迭代后就冲到了 4108.44，从第 5 轮开始就基本稳定在 4600~4670 的区间里，最后算下来差不多是 BC-only 的 4.7 倍。

是否达到专家水平。DAgger 在 5 轮之内就已经接近专家水平了，第 5 轮 4619.60，第 9 轮 4593.69，和专家回报 4681.89 的差距基本在 2% 左右。进步最明显的是第 0 轮到第 1 轮之间，一下子从 976.30 跳到了 4108.44，后面因为策略已经进入了高回报的状态区域，提升就变得比较平缓了。

分布漂移的影响与 DAgger 的解决方式。BC 出问题主要是因为分布漂移。策略在测试时碰到的是专家演示里没出现过的状态，在 Ant 上这些没见过的情况会让误差越攒越多，智能体很容易失控到没法恢复的局面，所以回报只有专家的 21%。DAgger 的做法其实挺直观的，分三步走：（1）先用当前策略在环境里跑，看看它到底会进哪些状态；（2）让专家给这些状态重新标上正确的动作；（3）把新数据加进训练集。这样训练分布就能跟上测试时真正遇到的情况，策略就算碰到专家以前没展示过的状态也能学怎么应对。

4 提交说明：仅提交 PDF 报告

完成实验后，在 `hw1_template.tex` 中填写答案、实验设置、结果表格、曲线和分析，并在报告中注明姓名与学号。所有必需的实验结果均应包含在报告内。

使用 XeLaTeX 编译报告。在模板所在目录运行：

```
xelatex hw1_template.tex
xelatex hw1_template.tex
```

生成文件为 `hw1_template.pdf`。

如使用 Overleaf，请选择 XeLaTeX 编译器，以 `hw1_template.tex` 为主文档，编译后下载 PDF。

最终只需上传填写完整的 PDF 报告，无需上传代码、实验日志、视频、TeX 源文件或 ZIP 压缩包。代码与日志请自行保留，供生成图表与核查结果使用。提交前请打开 PDF，检查图表、公式及答案是否显示完整。

提交入口：`https://oc.sjtu.edu.cn/courses/95065`。截止日期以课程平台通知为准。

5 Visualization (Optional)

Use TensorBoard to inspect your training logs:

```
tensorboard --logdir data
```

You will see scalar summaries (e.g., `Eval_AverageReturn`, `Training Loss`) as well as videos of your trained policies (in the "Images" tab, if video logging was enabled).

To compare specific runs:

```
tensorboard --logdir data/run1,data/run2,data/run3
```

You can also use the provided `scripts/run_hw1.ipynb` notebook to extract data from TensorBoard event files and produce publication-quality plots.

6 Survey

Please estimate, in minutes, for each section, how much time you spent (a) writing code and (b) waiting for the results. This will help us calibrate the difficulty for future homeworks.

- **Behavior Cloning (coding):** 50 分钟
- **Behavior Cloning (experiments):** 100 分钟
- **DAgger (coding):** 25 分钟
- **DAgger (experiments):** 60 分钟